
SYNTRIX

CLOUDPULSE

Complete Project Architecture & Implementation Guide

Reverse-Engineered Technical Documentation

Project	CloudMart Enterprise Observability Platform
Team	SYNTRIX
Purpose	Hackathon Project — CTS Cloud Innovation Challenge
Document Type	Reverse-Engineered Architecture & Implementation Guide
Version	1.0
Generated	12 September 2026, 09:54 IST
GCP Project ID	project-4e3f1563-833a-4721-bf7
Region	asia-south1 (Mumbai)

Table of Contents

1. Executive Summary
2. Project Objective
3. Actual Final Architecture
4. Technology & Service Inventory
5. Application Architecture
6. Build From Zero — Chronological Roadmap
7. Containerization
8. Artifact Registry
9. Cloud Run
10. GKE (Google Kubernetes Engine)
11. Compute Engine
12. Cloud SQL
13. Logging Architecture
14. Log Streamer Daemon
15. BigQuery
16. ETL & Analytics
17. Cloud Monitoring
18. Alerting
19. Looker Studio
20. Cost Optimization / FinOps
21. IAM & Security
22. Networking
23. End-to-End Runtime Flow
24. Configuration Master Table
25. Command Reference
26. Design Decisions & Trade-offs
27. Project Limitations
28. What Is Actually Implemented vs Conceptual
29. Presentation Talking Points
30. Judge Q&A;
31. Personal Learning Roadmap
32. Final Cheat Sheet
- Appendix

1. Executive Summary

CloudPulse (codename: CloudMart) is a cloud-native e-commerce observability platform built by Team SYNTRIX for the CTS Cloud Innovation Hackathon. The project demonstrates a complete observability pipeline: from application microservices generating structured JSON logs, through Google Cloud's native logging infrastructure, into BigQuery for analytics, and finally visualized in Looker Studio dashboards — with Cloud Monitoring alerting layered on top.

The system is architected across **three distinct GCP compute platforms** (Cloud Run, GKE, Compute Engine) to demonstrate multi-environment observability. A custom **Log Streamer Daemon** bridges Google Cloud Logging to BigQuery via API polling and NDJSON batch loading. Six BigQuery analytical views power Looker Studio dashboards covering latency, errors, usage, performance KPIs, and cost optimization recommendations.

■ **Key Innovation:** Unlike simple log-forwarding projects, CloudPulse implements a full-cycle observability pipeline with custom ETL, cross-platform log correlation via distributed tracing, simulation-driven chaos engineering for generating organic error telemetry, and SQL-based FinOps recommendations — all orchestrated across three compute environments.

At a Glance

Metric	Value
Microservices	6 (Gateway + 5 backend services)
Compute Platforms	3 (Cloud Run, GKE, Compute Engine)
Database	Cloud SQL PostgreSQL 15 (db-f1-micro)
Log Pipeline	Cloud Logging → Custom Streamer → BigQuery
Analytics Views	6 (cleaned logs + 5 analytical)
Alert Policies	5 (Cloud Run, GKE, GCE, BigQuery, Pipeline)
Notification Channels	7 email channels (team members)
Event System	Pub/Sub (order-events, payment-events, inventory-events)
Frontend	SPA with glassmorphism UI + Developer Control Panel
Region	asia-south1 (Mumbai)

2. Project Objective

Build a production-style cloud-native application that generates rich, organic, structured JSON telemetry and demonstrates a complete observability pipeline on Google Cloud Platform.

Primary Goals

- Demonstrate multi-platform deployment (Cloud Run + GKE + Compute Engine) of microservices
- Implement structured JSON logging with distributed tracing across all compute environments
- Build a custom log pipeline from Cloud Logging to BigQuery for analytical workloads
- Create analytical views and Looker Studio dashboards for operational intelligence
- Implement Cloud Monitoring alert policies for proactive incident detection
- Demonstrate FinOps/cost optimization through SQL-based traffic analysis
- Show simulation-driven chaos engineering for generating realistic error telemetry

Secondary Goals

- Event-driven architecture via Pub/Sub for order, payment, and inventory events
- Kubernetes secrets management for sensitive database credentials
- IAM least-privilege service accounts for Cloud Run workloads
- VPC-based internal networking between Cloud Run, GKE, and Cloud SQL

3. Actual Final Architecture

The following architecture was reverse-engineered from the actual repository source code, deployment scripts, Kubernetes manifests, Cloud Run configurations, and BigQuery SQL. Every component listed below has direct evidence in the codebase.

3.1 Application & Data Flow Architecture

USER (Browser)
■ HTTPS
↓
FastAPI Gateway ← Cloud Run (syntrix-gateway)
■ Serves SPA frontend (index.html, app.js, admin.js)
■ Proxies /api/* requests to backend services
■
■■■■ /api/products, /api/categories, /api/search
■ → Catalog Service ← Cloud Run (syntrix-catalog)
■
■■■■ /api/cart, /api/cart/items
■ → Cart Service ← Cloud Run (syntrix-cart)
■
■■■■ /api/checkout, /api/orders
■ → Order Service ← GKE (order-deployment)
■ ■■■■ calls Cart Service (validate cart)
■ ■■■■ calls Inventory Service (reserve stock)
■ ■■■■ calls Payment Service (process payment)
■ ■■■■ publishes to Pub/Sub (order-events)
■
■■■■ /api/payments
■ → Payment Service ← GKE (payment-deployment)
■ ■■■■ publishes to Pub/Sub (payment-events)
■
■■■■ /api/inventory
→ Inventory Service ← Compute Engine (syntrix-inventory)
■■■■ publishes to Pub/Sub (inventory-events)
ALL SERVICES → Cloud SQL PostgreSQL (syntrix-postgres)

3.2 Observability Pipeline Architecture

ALL SERVICES (Cloud Run / GKE / Compute Engine)

■ Structured JSON logs → stdout

↓

Google Cloud Logging

■ Automatic ingestion by platform

↓

Log Streamer Daemon (realtime_log_streamer.py)

■ Polls via: gcloud logging read (every 10 seconds)

■ Filters: resource.type = cloud_run_revision | gce_instance | k8s_container

■ Sanitizes keys, stringifies nested JSON

■ Writes batch to /tmp/stream_batch.ndjson

↓

BigQuery (syntrix_logs.raw_logs)

■ Loaded via: bq load --source_format=NEWLINE_DELIMITED_JSON

↓

BigQuery Views (ETL)

■■■ lookout_cleaned_logs (base cleaning + field extraction)

■■■ lookout_latency_analysis (avg/max latency by service/endpoint)

■■■ lookout_error_analysis (4xx/5xx rates by service)

■■■ lookout_usage_monitoring (daily request counts by endpoint)

■■■ lookout_performance_kpis (p50/p90/p99 latency, success rate)

■■■ lookout_cost_optimization (traffic-based cost recommendations)

↓

Looker Studio Dashboards (6 sheets)

PARALLEL: Cloud Monitoring

■■■ Alert: Cloud Run 5xx Errors → Email (Babu, Mugunthan)

■■■ Alert: GKE High CPU → Email (Akash)

■■■ Alert: GCE High CPU → Email (Roshni, Swathi)

■■■ Alert: BigQuery High Execution Time → Email (Ecclesiastes, Emayan)

■■■ Alert: Log Pipeline Errors → Email (Varunshiyam)

[CONFIRMED] Architecture verified from: gateway/main.py, deploy_*.sh, deploy_gke.sh K8s manifests, realtime_log_streamer.py, bq_views.sql, create_alerts.sh

4. Technology & Service Inventory

Classification of every Google Cloud service based on evidence found in the repository:

Service	Status	Evidence
Cloud Run	ACTUALLY IMPLEMENTED	Gateway, Catalog, Cart deployed as Cloud Run services; Seed as Cloud Run Job
Artifact Registry	ACTUALLY IMPLEMENTED	syntrix-cloudmart repository in asia-south1-docker.pkg.dev
GKE (Autopilot)	ACTUALLY IMPLEMENTED	syntrix-gke cluster; Order + Payment deployments with LoadBalancer services
Compute Engine	ACTUALLY IMPLEMENTED	syntrix-inventory VM (e2-micro) in asia-south1-a with Docker startup script
Cloud Logging	ACTUALLY IMPLEMENTED	Automatic ingestion from all platforms; structured JSON via python-json-logger
Cloud Monitoring	ACTUALLY IMPLEMENTED	5 alert policies created via create_alerts.sh and fix_alerts.py
BigQuery	ACTUALLY IMPLEMENTED	syntrix_logs dataset with raw_logs table + 6 analytical views
Looker Studio	ACTUALLY IMPLEMENTED	Dashboard design documented in Looker_Dashboard_Design.md (1143 lines)
Cloud SQL	ACTUALLY IMPLEMENTED	syntrix-postgres (PostgreSQL 15, db-f1-micro) in asia-south1
Pub/Sub	ACTUALLY IMPLEMENTED	3 topics: order-events, payment-events, inventory-events; publisher in code
Secret Manager	ACTUALLY IMPLEMENTED	database-url secret created in setup_cloudsql.sh
IAM	ACTUALLY IMPLEMENTED	cloudrun-app-sa service account, roles/cloudsql.client, secretmanager.secretAccessor
VPC Networking	ACTUALLY IMPLEMENTED	Default VPC, firewall rules, vpc-egress=all-traffic for Cloud Run
Cloud Storage	PARTIALLY IMPLEMENTED	gs://project-...-logs-bucket referenced in export_logs.sh for log staging
Log Router / Sink	NOT IMPLEMENTED	Logs are polled via gcloud CLI, not exported via native log sinks
Cloud Functions	NOT IMPLEMENTED	No evidence in repository
Cloud Scheduler	NOT IMPLEMENTED	No evidence in repository
API Gateway (managed)	NOT IMPLEMENTED	Custom FastAPI gateway used instead
Recommender API	NOT IMPLEMENTED	Cost recommendations are custom SQL logic, not GCP Recommender
Cloud Billing API	NOT IMPLEMENTED	No billing API integration found

Service	Status	Evidence
Load Balancing (explicit)	PARTIALLY IMPLEMENTED	GKE internal LoadBalancer services; no external HTTP(S) LB

Application Technology Stack

Component	Technology
Language	Python 3.12
Web Framework	FastAPI + Uvicorn
ORM	SQLAlchemy
Database	PostgreSQL 15
HTTP Client	httpx (async)
Logging	python-json-logger (pythonjsonlogger)
Container Runtime	Docker (python:3.12-slim)
Frontend	Vanilla HTML/CSS/JS (SPA)
Pub/Sub Client	google-cloud-pubsub
Migrations	Alembic (configured but migrations not generated)
Testing	pytest + httpx (async)

5. Application Architecture

CloudMart is a microservices-based e-commerce platform consisting of 6 services (1 gateway + 5 backend) that communicate via HTTP REST. All services share a common PostgreSQL database via SQLAlchemy ORM and emit structured JSON logs via a shared logging library.

5.1 Service Inventory

Service	Entry Point	Frame work	Platform	Port	Responsibilities	Dependencies
Gateway	gateway/main.py	FastAPI	Cloud Run	8080	API proxy, frontend hosting, simulation config	Catalog, Cart, Order, Payment, Inventory (via HTTP proxy)
Catalog	services/catalog/main.py	FastAPI	Cloud Run	8080	Product listing, search, categories	PostgreSQL (read-only)
Cart	services/cart/main.py	FastAPI	Cloud Run	8080	Cart CRUD, item management	PostgreSQL (read-write)
Order	services/order/main.py	FastAPI	GKE	8080	Checkout orchestration, order lifecycle	Cart, Inventory, Payment (HTTP); Pub/Sub; PostgreSQL
Payment	services/payment/main.py	FastAPI	GKE	8080	Payment processing, simulation modes	Pub/Sub; PostgreSQL
Inventory	services/inventory/main.py	FastAPI	Compute Engine	8000	Stock management, reserve/release	Pub/Sub; PostgreSQL

5.2 Service Dependency Map

Gateway (Cloud Run)
■■■ Catalog Service (Cloud Run) – product browsing
■■■ Cart Service (Cloud Run) – cart management
■■■ Order Service (GKE) – checkout orchestration
■ ■■■ Cart Service – validates cart contents
■ ■■■ Inventory Service (GCE) – reserves stock
■ ■■■ Payment Service (GKE) – processes payment
■ ■■■ Pub/Sub (order-events) – publishes order.created
■■■ Payment Service (GKE) – direct payment queries
■ ■■■ Pub/Sub (payment-events) – publishes payment.completed/failed
■■■ Inventory Service (Compute Engine) – stock queries
■■■ Pub/Sub (inventory-events) – publishes inventory.reserved/insufficient

[CONFIRMED] Verified from: gateway/main.py SERVICES map, order/main.py HTTP calls, payment/main.py and inventory/main.py Pub/Sub publishers

5.3 API Endpoint Reference

Method	Endpoint	Route	Description
GET	/api/products	Gateway → Catalog	List all products
GET	/api/products/{id}	Gateway → Catalog	Get single product
GET	/api/categories	Gateway → Catalog	List categories
GET	/api/search?q=	Gateway → Catalog	Search products
POST	/api/cart	Gateway → Cart	Create new cart
GET	/api/cart/{id}	Gateway → Cart	Get cart contents
POST	/api/cart/items	Gateway → Cart	Add item to cart
PUT	/api/cart/items/{id}	Gateway → Cart	Update item quantity
DELETE	/api/cart/items/{id}	Gateway → Cart	Remove item
POST	/api/checkout	Gateway → Order	Full checkout flow
GET	/api/orders	Gateway → Order	List all orders
GET	/api/orders/{id}	Gateway → Order	Get single order
POST	/api/payments	Gateway → Payment	Process payment
GET	/api/payments/{id}	Gateway → Payment	Get payment
GET	/api/inventory	Gateway → Inventory	List inventory
POST	/api/inventory/reserve	Gateway → Inventory	Reserve stock
POST	/api/inventory/release	Gateway → Inventory	Release stock
GET/POST	/api/simulation	Gateway (direct)	Simulation config
GET	/health	All services	Health check

5.4 Database Schema

All services share a single PostgreSQL database (cloudmart) with the following tables, defined in `shared/models/models.py` using SQLAlchemy declarative_base:

Table	Columns	Purpose
products	id, sku, name, category, price, stock, rating	Product catalog, seeded with 50 items
customers	id, name, email	Customer records (1 seeded: John Doe)
carts	id, customer_id (FK→customers)	Shopping cart containers
cart_items	id, cart_id (FK→carts), product_id (FK→products), quantity	Cart line items
orders	id, customer_id (FK→customers), total_amount, payment_status, order_status, created_at	Order records

Table	Columns	Purpose
payments	id, order_id (FK→orders), amount, status, created_at	Payment transaction records
inventory_transactions	id, product_id (FK→products), quantity, transaction_type, created_at	Stock movement audit log
simulation_config	key (PK), value	Runtime simulation settings (payment_mode, inventory_mode, db_mode)

5.5 Simulation / Chaos Engineering

The application includes a built-in chaos engineering system controlled via the Developer Control Panel in the frontend footer. Settings are stored in the simulation_config table and checked at runtime.

Config Key	Possible Values	Effect
payment_mode	normal 20_timeout 50_timeout declined	Simulates payment failures: 20% timeout, 50% timeout, or 30% declined
inventory_mode	healthy low_stock out_of_stock	Simulates inventory issues: forces stock to 2 or refuses all reservations
db_mode	normal slow_queries	Injects 1.5 second delay into all database operations

The frontend also includes a **Traffic Simulator** that can generate 10, 50, 100, or 250 concurrent simulated users performing browse → add-to-cart → checkout flows. This is how organic, realistic log data is generated for the observability pipeline.

6. Build From Zero — Chronological Roadmap

Reconstructed build sequence based on repository evidence, script dependencies, and deployment order:

Step 1: Application Architecture Design

- **What:** Designed 6-service microservice architecture with shared libraries
- **Detail:** Created: `shared/logging/logger.py`, `shared/middleware/tracing.py`, `shared/models/models.py`, `shared/utis/database.py`
- **Files:** `gateway/main.py`, `services/*/main.py`

Step 2: Local Development Environment

- **What:** Built `docker-compose.yml` for local development with PostgreSQL + all services
- **Detail:** All 6 services run locally on port 8000 behind the gateway; seed data populates 50 products
- **Files:** `docker-compose.yml`, `database/seed_data/seed.py`

Step 3: Frontend Development

- **What:** Built glassmorphism SPA with dashboard, catalog, cart, checkout, orders views
- **Detail:** Developer Control Panel for chaos engineering; Traffic Simulator for load generation
- **Files:** `frontend/index.html`, `frontend/app.js`, `frontend/admin.js`, `frontend/styles.css`

Step 4: Containerization

- **What:** Created multi-service Dockerfile using `SERVICE_NAME` build arg for image reuse
- **Detail:** Single Dockerfile builds any service by changing the build argument
- **Files:** `Dockerfile`, `Dockerfile.seed`

Step 5: Cloud SQL Setup

- **What:** Provisioned Cloud SQL PostgreSQL 15 (db-f1-micro) in `asia-south1`
- **Detail:** Created database, user, Secret Manager secret, service account with `cloudsql.client` role
- **Files:** `setup_cloudsql.sh`

Step 6: Artifact Registry

- **What:** Created `syntrix-cloudmart` repository for Docker images
- **Detail:** All images tagged and pushed to `asia-south1-docker.pkg.dev`
- **Files:** `deploy_phase1.sh` (`gcloud auth configure-docker`)

Step 7: Cloud Run Deployment (Phase 1)

- **What:** Deployed gateway, cart, catalog as Cloud Run services; seed as Cloud Run Job
- **Detail:** Allow-unauthenticated access, Cloud SQL connector, environment variables

-
- **Files:** `deploy_phase1.sh`, `update_cloudrun_sql.sh`, `setup_seed.sh`

Step 8: GKE Cluster & Deployment

- **What:** Created Autopilot cluster `syntrix-gke`; deployed order + payment services
- **Detail:** Kubernetes Deployments (1 replica each), LoadBalancer Services (internal), db-secret for `DATABASE_URL`
- **Files:** `setup_gke_cluster.sh`, `deploy_gke.sh`

Step 9: Compute Engine Deployment

- **What:** Created e2-micro VM `syntrix-inventory` with Docker startup script
- **Detail:** Pulls image from Artifact Registry, gets `DB_URL` from instance metadata, firewall rule for internal traffic
- **Files:** `deploy_inventory.sh`, `startup.sh`

Step 10: Gateway Integration

- **What:** Updated gateway with actual service URLs (Cloud Run URLs, GKE internal IPs, GCE internal IP)
- **Detail:** VPC egress for Cloud Run → internal services, identity token for Cloud Run → Cloud Run auth
- **Files:** `update_cloudrun_sql.sh`

Step 11: Structured Logging Implementation

- **What:** Implemented `CustomJsonFormatter` with Cloud Logging integration fields
- **Detail:** Fields: `timestamp`, `severity`, `service`, `environment`, `request_id`, `trace_id`, `logging.googleapis.com/trace`
- **Files:** `shared/logging/logger.py`, `shared/middleware/tracing.py`

Step 12: Pub/Sub Event System

- **What:** Added Pub/Sub publishers to Order, Payment, and Inventory services
- **Detail:** Topics: `order-events`, `payment-events`, `inventory-events`; events published on key business actions
- **Files:** `services/order/main.py`, `services/payment/main.py`, `services/inventory/main.py`

Step 13: Log Export & BigQuery Load

- **What:** Created `export_logs.sh` for initial bulk log extraction + `prepare_bq_json.py` for sanitization
- **Detail:** Reads from Cloud Logging, converts to NDJSON, uploads to GCS, loads into BigQuery
- **Files:** `export_logs.sh`, `prepare_bq_json.py`

Step 14: Realtime Log Streamer

- **What:** Created Python daemon for continuous log polling and BigQuery loading
- **Detail:** 10-second polling interval, 2000-log batch limit, key sanitization, incremental timestamp tracking
- **Files:** `realtime_log_streamer.py`

Step 15: BigQuery Analytics Views

- **What:** Created 6 analytical views for Looker Studio consumption
- **Detail:** ETL from raw_logs → cleaned → latency/error/usage/KPI/cost analysis
- **Files:** bq_views.sql

Step 16: Cloud Monitoring Alerts

- **What:** Created 5 alert policies with 7 notification channels (email)
- **Detail:** Cloud Run 5xx, GKE CPU, GCE CPU, BigQuery execution time, Pipeline errors
- **Files:** create_alerts.sh, fix_alerts.py, add_mugunthan.py

Step 17: Looker Studio Dashboard Design

- **What:** Designed 6-sheet dashboard with complete implementation guide
- **Detail:** Executive overview, performance, errors, infrastructure, cost optimization, log explorer
- **Files:** Looker_Dashboard_Design.md, Looker_Dashboard_Design.pdf

Step 18: Traffic Simulation & Testing

- **What:** Created test scripts and traffic simulators for organic log generation
- **Detail:** simulate_traffic.sh, inject_gce_logs.sh, test_checkout.py, test_integration.py
- **Files:** simulate_traffic.sh, test_*.py

7. Containerization

The project uses a single, parameterized Dockerfile that builds any service by changing the `SERVICE_NAME` build argument. This is an elegant approach that avoids Dockerfile duplication.

Dockerfile Analysis

- **Base Image:** `python:3.12-slim` — minimal Python image for small container size
- **Build Arg:** `SERVICE_NAME` — determines which service code is copied (e.g., `services/cart`, `gateway`)
- **Shared Library:** `shared/` is always copied to `/app/shared` (logging, middleware, models, DB utils)
- **Frontend:** `frontend/` is always copied (required by `gateway`, harmless for other services)
- **Port:** `EXPOSE 8000`; actual port set via `PORT` env var (default 8080 in CMD)
- **Workers:** 2 Uvicorn workers per container
- **PYTHONPATH:** Set to `/app` for proper import resolution

Build Command Pattern

```
docker buildx build --platform linux/amd64 \  
-t asia-south1-docker.pkg.dev/PROJECT_ID/syntrix-cloudmart/SERVICE:VERSION \  
--build-arg SERVICE_NAME=services/cart --push .
```

■ **Why `--platform linux/amd64`?** The development machine is macOS (likely Apple Silicon/ARM). GCP Cloud Run and GKE nodes run on AMD64/x86_64. Cross-platform build is required.

8. Artifact Registry

All container images are stored in Google Artifact Registry.

Property	Value
Repository	syntrix-cloudmart
Location	asia-south1-docker.pkg.dev
Full Path	asia-south1-docker.pkg.dev/project-4e3f1563-833a-4721-bf7/syntrix-cloudmart
Format	Docker
Auth Config	gcloud auth configure-docker asia-south1-docker.pkg.dev

Known Image Tags

Image	Tags Found
gateway	1.0.0
cart	1.0.0
catalog	Not explicit in scripts (deployed via Cloud Run)
order	1.0.1, 1.0.2
payment	1.0.1, 1.0.2
inventory	1.0.0
seed	1.0.0

[INFERRED] Order and Payment images have multiple versions (1.0.1, 1.0.2), suggesting at least one re-deployment iteration — likely to fix database connectivity or environment configuration issues.

9. Cloud Run

Three services + one job are deployed to Cloud Run:

Service	Image	Region	Env Vars (key ones)	Auth	Config
syntrix-gateway	asia-south1-docker.pkg.dev/.../gateway:1.0.0	asia-south1	SERVICE_NAME=gateway, CATALOG_URL=https://syntrix-catalog-..., CART_URL=https://syntrix-cart-..., ORDER_URL=http://10.160.0.5:8000, PAYMENT_URL=http://10.160.0.7:8000, INVENTORY_URL=http://10.160.0.6:8000, DATABASE_URL=postgresql://cloudmart_user:...@cloudmart?host=/cloudsql/...	allUsers (unauthenticated)	Cloud SQL connector, VPC egress=all-traffic, network=default
syntrix-catalog	asia-south1-docker.pkg.dev/.../catalog:*	asia-south1	SERVICE_NAME=catalog-service, DATABASE_URL=...	allUsers (unauthenticated)	Cloud SQL connector
syntrix-cart	asia-south1-docker.pkg.dev/.../cart:1.0.0	asia-south1	SERVICE_NAME=cart-service, DATABASE_URL=...	allUsers (unauthenticated)	Cloud SQL connector

Cloud Run Job: syntrix-seed

- **Image:** asia-south1-docker.pkg.dev/.../seed:1.0.0
- **Purpose:** One-time database schema creation + seed 50 products + default simulation config
- **VPC:** vpc-egress=all-traffic, network=default (required to reach Cloud SQL)

■ **Critical Architecture Detail:** The gateway connects to GKE and Compute Engine services via **internal IP addresses** (10.160.0.5, 10.160.0.6, 10.160.0.7), not via public URLs. This requires VPC egress configuration on Cloud Run (--vpc-egress=all-traffic --network=default). The gateway also connects to other Cloud Run services (catalog, cart) via their public HTTPS URLs with identity token authentication.

10. GKE (Google Kubernetes Engine)

10.1 Cluster Configuration

Property	Value
Cluster Name	syntrix-gke
Type	Autopilot (created via create-auto)
Region	asia-south1
Node Management	Fully managed by Google (Autopilot)
Image Pull Secret	gcr-json-key (referenced in manifests)

10.2 Workloads

Order Service Deployment

Property	Value
Kind	Deployment
Name	order-deployment
Replicas	1
Container Image	asia-south1-docker.pkg.dev/.../order:1.0.2
Container Port	8080
DATABASE_URL	From Kubernetes Secret (db-secret)
Service Type	LoadBalancer (Internal)
Service Port	8000 → 8080 (targetPort)
Selector	app: order

Payment Service Deployment

Property	Value
Kind	Deployment
Name	payment-deployment
Replicas	1
Container Image	asia-south1-docker.pkg.dev/.../payment:1.0.2
Container Port	8080
DATABASE_URL	From Kubernetes Secret (db-secret)
Service Type	LoadBalancer (Internal)

Property	Value
Service Port	8000 → 8080 (targetPort)
Selector	app: payment

■ **Why GKE for Order & Payment?** These services handle complex orchestration (Order calls Cart → Inventory → Payment → Pub/Sub) and financial transactions. GKE provides persistent pods, direct internal networking, and the ability to demonstrate Kubernetes-native operations. It also creates a distinct resource.type (k8s_container) in Cloud Logging for multi-platform observability demonstrations.

Database Secret: The DATABASE_URL is stored as a Kubernetes Secret (db-secret) created via `kubectl create secret generic`. The URL points directly to the Cloud SQL instance's public IP (34.14.131.84) with `cloudmart_user` credentials.

[INFERRED] GKE connects to Cloud SQL via public IP (34.14.131.84), not via Cloud SQL Auth Proxy. This was likely a pragmatic hackathon decision for simplicity.

11. Compute Engine

Property	Value
VM Name	syntrix-inventory
Machine Type	e2-micro (0.25 vCPU, 1 GB RAM)
Zone	asia-south1-a
Tags	http-server
Scopes	cloud-platform (full access)
Startup Script	Installs Docker, pulls inventory image, runs container
DATABASE_URL	Passed via instance metadata (database-url attribute)
Container	inventory:1.0.0 running on port 8000
Firewall Rule	allow-inventory-internal: tcp:8080 from 10.0.0.0/8

■ **Why Compute Engine for Inventory?** This demonstrates a legacy/stateful workload pattern. The Inventory service simulates a component that might exist on a traditional VM in a real enterprise migration. It creates `gce_instance` resource type in Cloud Logging, completing the three-platform observability story.

The startup script retrieves the database URL from GCE instance metadata using the metadata server (<http://metadata.google.internal/computeMetadata/v1/instance/attributes/database-url>). This avoids hardcoding credentials in the startup script.

12. Cloud SQL

Property	Value
Instance Name	syntrix-postgres
Engine	PostgreSQL 15
Tier	db-f1-micro (shared vCPU, 0.6 GB RAM)
Region	asia-south1
Database Name	cloudmart
User	cloudmart_user
Password	syntrix_admin_123 (hardcoded in deploy scripts; originally generated via openssl)
Connection Name	project-4e3f1563-833a-4721-bf7:asia-south1:syntrix-postgres
Tables	8 (products, customers, carts, cart_items, orders, payments, inventory_transactions, simulation_config)

Connectivity Patterns

Platform	Connection Method	Status
Cloud Run	Cloud SQL Auth Proxy (Unix socket via --add-cloudsql-instances)	CONFIRMED
GKE	Direct TCP to public IP (34.14.131.84:5432)	CONFIRMED
Compute Engine	Direct TCP to public IP (via metadata-derived URL)	CONFIRMED
Local Dev	Docker Compose PostgreSQL on localhost:5432	CONFIRMED

13. Logging Architecture

CloudMart implements structured JSON logging using `python-json-logger`. Every service emits logs to `stdout`, which are automatically captured by the respective compute platform's logging agent and forwarded to Google Cloud Logging.

13.1 Log Format

Each log entry contains the following fields (from `CustomJsonFormatter` in `shared/logging/logger.py`):

Field	Value	Presence
timestamp	UTC ISO 8601 (e.g., 2026-08-25T13:45:12.123Z)	Always
severity	INFO, WARNING, ERROR	Always
message	Human-readable log message	Always
service	From <code>SERVICE_NAME</code> env var	Always
environment	From <code>ENVIRONMENT</code> env var (local/cloud)	Always
request_id	UUID from <code>X-Request-Id</code> header or auto-generated	When request context exists
trace_id	UUID from <code>X-Trace-Id</code> header or auto-generated	When request context exists
logging.googleapis.com/trace	projects/PROJECT_ID/traces/TRACE_ID	When trace_id exists
logging.googleapis.com/spanId	First 16 chars of trace_id	When trace_id exists
event	Business event type (e.g., <code>order_created</code> , <code>payment_processed</code>)	Application logs
endpoint	HTTP path	Request completion logs
method	HTTP method	Request completion logs
status_code	HTTP response code	Request completion logs
latency_ms	Request processing time in milliseconds	Request completion logs

13.2 Distributed Tracing

Distributed tracing is implemented via Python `ContextVars`. The `TracingMiddleware` (`shared/middleware/tracing.py`) extracts or generates `X-Request-Id` and `X-Trace-Id` headers on each request, stores them in thread-local context variables, and injects them into all downstream HTTP calls. This creates a correlation chain:

```
Gateway receives request
→ Generates/extracts request_id and trace_id
→ Stores in ContextVars (request_id_var, trace_id_var)
→ Injects into outgoing HTTP headers
→ Each downstream service extracts the same IDs
→ All logs from the same user request share the same trace_id
```

→ Cloud Logging correlates them via `logging.googleapis.com/trace`

[CONFIRMED] Verified: ContextVars in `logger.py`, TracingMiddleware in `tracing.py`, header propagation in `gateway/main.py` and `order/main.py`

14. Log Streamer Daemon

The Log Streamer (`realtime_log_streamer.py`) is a custom Python daemon that bridges Google Cloud Logging to BigQuery. It was built because native Log Sinks to BigQuery were not used in this architecture.

Property	Value
Polling Method	gcloud logging read (subprocess)
Poll Interval	10 seconds
Batch Size	Up to 2000 log entries per poll
Filter	<code>resource.type=("cloud_run_revision" OR "gce_instance" OR "k8s_container")</code>
Deduplication	Incremental timestamp tracking (<code>last_timestamp + 1 nanosecond</code>)
Output Format	NDJSON (Newline-Delimited JSON)
Temp File	<code>/tmp/stream_batch.ndjson</code>
BigQuery Load	<code>bq load --source_format=NEWLINE_DELIMITED_JSON --schema_update_option=ALLOW_FIELD_ADDITION</code>
Target Table	<code>project-...:syntrix_logs.raw_logs</code>
Key Sanitization	Replaces non-alphanumeric chars with underscores (BigQuery column name requirement)
Nested JSON Handling	Stringifies labels, resource, jsonPayload, protoPayload, sourceLocation, operation fields
Initial Window	60 minutes before daemon start
Error Handling	Continues polling on failure; prints error to stdout

Why Custom Streamer Instead of Native Log Sink?

- Log Sinks export raw Cloud Logging format which requires complex BigQuery schema management
- The custom streamer sanitizes field names and stringifies nested JSON for clean BigQuery loading
- Provides more control over data transformation before loading

[INFERRED] This was likely also a practical hackathon decision — the team may have encountered schema issues with native sinks and pivoted to a custom solution.

Why NDJSON?

BigQuery's native batch loading format is NDJSON (Newline-Delimited JSON). Each line is a complete JSON object representing one log entry. This format enables streaming-compatible loading with `--schema_update_option=ALLOW_FIELD_ADDITION`, which automatically adds new columns as log schemas evolve.

15. BigQuery

15.1 Dataset & Table

Property	Value
Dataset	syntrix_logs
Raw Table	raw_logs
Load Method	bq load (NDJSON, schema auto-detect, ALLOW_FIELD_ADDITION)
Partitioning	Not explicitly configured (auto-detect)
Clustering	Not explicitly configured

15.2 Raw Log Files

- **raw_logs.json:** 1.2 MB — full JSON array export from Cloud Logging
- **raw_logs.ndjson:** 1.1 MB — sanitized NDJSON version for BigQuery loading

16. ETL & Analytics

Six BigQuery views transform raw logs into analytical datasets (defined in `bq_views.sql`):

Base ETL View — `looker_cleaned_logs`

- **Description:** Extracts structured fields from raw Cloud Logging data. Determines `compute_environment` from `resource.type`, derives `service_name` based on platform (Cloud Run → `service_name` label, GKE → `container_name`, GCE → hardcoded 'inventory-service'). Extracts `endpoint`, `method`, `status_code`, `latency_ms`, `event_type`, `error_message` from `jsonPayload`.
- **Key Logic:** Filters to logs where `event` or `latency_ms` is present (app-level logs only)
- **Business Purpose:** Foundation for all downstream views

Latency Analysis — `looker_latency_analysis`

- **Description:** Aggregates by hour, service, compute environment, and endpoint. Calculates `total_requests`, `avg_latency_ms`, `max_latency_ms`.
- **Key Logic:** WHERE `latency_ms` IS NOT NULL; GROUP BY `TIMESTAMP_TRUNC(timestamp, HOUR)`
- **Business Purpose:** Identifies slow services and endpoints; supports SRE latency SLO monitoring

Error Analysis — `looker_error_analysis`

- **Description:** Counts client errors (4xx), server errors (5xx), and calculates `error_rate_percentage` per hour/service/environment.
- **Key Logic:** `SAFE_DIVIDE(COUNTIF(status_code >= 400), COUNT(1)) * 100`
- **Business Purpose:** Error trend monitoring; identifies reliability issues by service and platform

Usage Monitoring — `looker_usage_monitoring`

- **Description:** Daily request counts by compute environment and endpoint.
- **Key Logic:** GROUP BY `TIMESTAMP_TRUNC(timestamp, DAY)`, `compute_environment`, `endpoint`
- **Business Purpose:** Traffic pattern analysis; identifies most/least used endpoints

Performance KPIs — `looker_performance_kpis`

- **Description:** Calculates latency percentiles (p50, p90, p99) and `success_rate_percentage` per service and environment.
- **Key Logic:** `APPROX_QUANTILES(latency_ms, 100)[OFFSET(50/90/99)]`
- **Business Purpose:** SRE golden signals dashboard; SLO compliance tracking

Cost Optimization — `looker_cost_optimization_recommendations`

- **Description:** Analyzes last-24-hour traffic relative to latest data timestamp. Generates cost recommendations based on request volume and compute platform.
- **Key Logic:** Rules: <100 reqs on GKE → 'Scale down or migrate to Cloud Run'; <100 on GCE → 'Downgrade VM'; >10000 on Cloud Run → 'Consider GKE for sustained discounts'

-
- **Business Purpose:** FinOps recommendations; right-sizing guidance

[CONFIRMED] All 6 views verified from bq_views.sql

17. Cloud Monitoring

Cloud Monitoring is used for alerting based on platform-native metrics. No custom metrics or dashboards were created in Cloud Monitoring — visualization is handled entirely by Looker Studio using BigQuery data.

■ **Logs vs Metrics in this project:** Cloud Logging captures detailed per-request data (JSON logs). Cloud Monitoring tracks platform-level metrics (CPU utilization, request counts, query times). The project uses logs for analytics (via BigQuery) and metrics for alerting (via Cloud Monitoring).

18. Alerting

Five alert policies are configured via `create_alerts.sh`:

Alert Name	Metric	Threshold	Duration	Notified
Cloud Run 5xx Errors	<code>run.googleapis.com/request_count (5xx class)</code>	> 10 requests	60s	Babu, Mugunthan
GKE High CPU	<code>kubernetes.io/container/cpu/limit_utilization</code>	> 80%	180s	Akash
GCE High CPU	<code>compute.googleapis.com/instance/cpu/utilization</code>	> 85%	180s	Roshni, Swathi
BigQuery High Exec Time	<code>bigquery.googleapis.com/query/execution_times</code>	> 10000ms	60s	Ecclesiastes, Emayan
Log Pipeline Errors	<code>logging.googleapis.com/log_entry_count (ERROR)</code>	> 5 errors	60s	Varunshiyam

Notification Channels

7 email notification channels were created, one per team member, each responsible for specific infrastructure:

Team Member	Email	Responsibility
Babu Senthil	<code>bbabusenthil@gmail.com</code>	Cloud Run
Mugunthan	<code>mt.mugunthan@gmail.com</code>	Cloud Run (added later via <code>add_mugunthan.py</code>)
Varunshiyam	<code>varunshiyam.analyst@gmail.com</code>	Looker Pipeline
Akash	<code>akaashrk02@gmail.com</code>	GKE
Roshni	<code>roshnigloriya@gmail.com</code>	GCE
Swathi	<code>swathi062005@gmail.com</code>	GCE
Ecclesiastes	<code>ecclesiastescherubin1@gmail.com</code>	BigQuery
Emayan	<code>emayanvijayakumar17@gmail.com</code>	BigQuery

Alert lifecycle: When a metric exceeds its threshold for the specified duration, Cloud Monitoring creates an Incident and sends email notifications to the configured channels. The `fix_alerts.py` script was created to add required aggregation parameters (`alignmentPeriod`, `perSeriesAligner`) that were missing from the initial alert policy definitions.

[INFERRED] The existence of `fix_alerts.py` suggests the initial `create_alerts.sh` had issues — alert policies without aggregation settings fail validation in newer Cloud Monitoring API versions.

19. Looker Studio

A comprehensive Looker Studio dashboard design is documented in `Looker_Dashboard_Design.md` (1,143 lines, 57KB). The dashboard is designed with 6 sheets (tabs) targeting different stakeholders.

Sheet	Title	Audience	Key Visualizations
Sheet 1	Executive Overview	CTO / VP Engineering	Total requests, error rate, top services, latency heatmap
Sheet 2	System Performance & Latency	SRE / DevOps	P50/P90/P99 latency, service comparison, slow endpoint detection
Sheet 3	Error Analysis & Reliability	SRE / DevOps	4xx/5xx breakdown, error trends, error rate by service
Sheet 4	Infrastructure & Resource Usage	FinOps / Cloud Mgmt	Request distribution by compute platform, usage trends
Sheet 5	Cost Optimization	FinOps / Cloud Mgmt	Cost recommendations, traffic-to-platform analysis
Sheet 6	Developer Deep Dive	Developers / QA	Log explorer, raw log inspection, trace correlation

[CONFIRMED] Dashboard design verified from `Looker_Dashboard_Design.md`. The PDF version also exists.

[INFERRED] The actual Looker Studio dashboard is assumed to be built in the GCP console based on this design document.

20. Cost Optimization / FinOps

Cost optimization is implemented via **custom SQL logic** in the `looker_cost_optimization_recommendations` BigQuery view — NOT via Google Cloud Recommender API.

Traffic Condition	Platform	Recommendation
< 100 requests/24h	k8s_container (GKE)	Scale down GKE nodes or migrate to Cloud Run (Scale to Zero)
< 100 requests/24h	gce_instance (GCE)	Downgrade VM instance type or migrate to Cloud Run
> 10000 requests/24h	cloud_run_revision	Consider migrating to GKE for sustained discount usage
All other cases	Any	Traffic Optimized for Environment

■ The view uses a self-referencing window (last 24h relative to `MAX(timestamp)` in the data, not `CURRENT_TIMESTAMP`), which ensures recommendations work correctly even with historical/older demo data.

21. IAM & Security

Service Account	Used By	Roles/Permissions	Purpose
cloudrun-app-sa	Cloud Run services	roles/cloudsql.client, roles/secretmanager.secretAccessor	Allows Cloud Run to connect to Cloud SQL via auth proxy and read database URL from Secret Manager
allUsers	Cloud Run services	roles/run.invoker	Allows unauthenticated public access to Cloud Run services. Required for public-facing gateway.
Default Compute SA	Compute Engine VM	cloud-platform scope	Broad access scope. Allows VM to pull images from Artifact Registry and write logs.
Default GKE SA	GKE Autopilot	Managed by Autopilot	Autopilot manages node service accounts automatically.

Security Concerns

- **Database password hardcoded** in deploy scripts (syntrix_admin_123) — acceptable for hackathon, not production
- **allUsers invoker** on all Cloud Run services — only gateway should be public
- **cloud-platform scope** on GCE VM — overly broad; should be scoped to specific APIs
- **Cloud SQL public IP** used by GKE — should use Cloud SQL Auth Proxy or private IP

22. Networking

Path	Protocol	Details
User → Gateway	HTTPS (public)	Public Cloud Run URL
Gateway → Catalog	HTTPS (public)	Cloud Run URL with identity token
Gateway → Cart	HTTPS (public)	Cloud Run URL with identity token
Gateway → Order	HTTP (internal VPC)	Internal IP 10.160.0.5:8000
Gateway → Payment	HTTP (internal VPC)	Internal IP 10.160.0.7:8000
Gateway → Inventory	HTTP (internal VPC)	Internal IP 10.160.0.6:8000
Cloud Run → Cloud SQL	Unix socket	Cloud SQL Auth Proxy (/cloudsql/CONNECTION_NAME)
GKE → Cloud SQL	TCP (public IP)	34.14.131.84:5432
GCE → Cloud SQL	TCP (public/internal IP)	Via metadata-derived URL
All → Cloud Logging	Automatic	Platform agent (stdout capture)

23. End-to-End Runtime Flow

This section traces a single checkout request through the entire architecture, showing both the data path and the observability path simultaneously.

Scenario: User Completes a Purchase

1. User clicks 'Confirm Order'

Browser sends POST `/api/checkout {cart_id: 5, customer_id: 1}` to Gateway

2. Gateway receives request

TracingMiddleware generates `request_id + trace_id`, stores in `ContextVars`. Gateway logs: `{event: request_completed, endpoint: /api/checkout}`. Proxies to Order Service at `http://10.160.0.5:8000/checkout`

3. Order Service: Validate Cart

Calls Cart Service (Cloud Run) GET `/cart/5`. Cart logs: `{event: request_completed}`. Returns cart items and total.

4. Order Service: Reserve Inventory

Calls Inventory Service (GCE) POST `/inventory/reserve`. Inventory decrements stock, creates `InventoryTransaction`, logs: `{event: inventory_reserved}`. Publishes to Pub/Sub `inventory-events: {event_type: inventory.reserved}`

5. Order Service: Create Order

Inserts Order record (`status=CREATED`, `payment=PENDING`) in Cloud SQL. Publishes to Pub/Sub `order-events: {event_type: order.created}`. Logs: `{event: order_created, order_id: 42}`

6. Order Service: Process Payment

Calls Payment Service (GKE) POST `/payments {order_id: 42, amount: 149.99}`. Payment checks `simulation_config`, processes payment, creates `Payment` record. Publishes to Pub/Sub `payment-events: {event_type: payment.completed}`. Logs: `{event: payment_processed, status: SUCCESS}`

7. Order Service: Finalize

Updates Order status to `COMPLETED`. Clears cart items. Logs: `{event: order_completed}`. Returns `{order_id: 42, status: COMPLETED}`

8. Gateway: Return Response

Forwards response to browser. Logs total gateway latency. All logs share the SAME `trace_id` for correlation.

Parallel Observability Path

- **Step 1:** All 6+ log entries (Gateway, Cart, Inventory, Order, Payment) are written to stdout as JSON
- **Step 2:** Cloud Run, GKE, GCE logging agents automatically forward to Google Cloud Logging
- **Step 3:** Log Streamer Daemon polls Cloud Logging every 10 seconds
- **Step 4:** Batch of logs sanitized and written to /tmp/stream_batch.ndjson
- **Step 5:** bq load appends to syntrix_logs.raw_logs in BigQuery
- **Step 6:** BigQuery views (looker_cleaned_logs → latency/error/usage/KPI) automatically reflect new data
- **Step 7:** Looker Studio dashboards show updated metrics on next refresh
- **Step 8:** If any metric exceeds threshold (e.g., 5xx errors > 10), Cloud Monitoring fires alert → email notification

24. Configuration Master Table

Component	Value	Source	Status
GCP Project ID	project-4e3f1563-833a-4721-bf7	All scripts	CONFIRMED
Region	asia-south1	All deployments	CONFIRMED
GKE Zone	asia-south1 (regional Autopilot)	setup_gke_cluster.sh	CONFIRMED
GCE Zone	asia-south1-a	deploy_inventory.sh	CONFIRMED
Artifact Registry	syntrix-cloudmart	All deploy scripts	CONFIRMED
Cloud SQL Instance	syntrix-postgres	setup_cloudsql.sh	CONFIRMED
Cloud SQL Tier	db-f1-micro	setup_cloudsql.sh	CONFIRMED
Database Name	cloudmart	setup_cloudsql.sh	CONFIRMED
Database User	cloudmart_user	deploy_gke.sh	CONFIRMED
GKE Cluster	syntrix-gke	setup_gke_cluster.sh	CONFIRMED
GKE Type	Autopilot	create-auto command	CONFIRMED
GCE Machine Type	e2-micro	deploy_inventory.sh	CONFIRMED
GCE VM Name	syntrix-inventory	deploy_inventory.sh	CONFIRMED
BigQuery Dataset	syntrix_logs	realtime_log_streamer.py	CONFIRMED
BigQuery Table	raw_logs	realtime_log_streamer.py	CONFIRMED
Log Poll Interval	10 seconds	realtime_log_streamer.py	CONFIRMED
Log Batch Limit	2000 entries	realtime_log_streamer.py	CONFIRMED
Pub/Sub Topics	order-events, payment-events, inventory-events	Service source code	CONFIRMED
Gateway URL	https://syntrix-gateway-541833001986.asia-south1.run.app	simulate_traffic.sh	CONFIRMED
Order Internal IP	10.160.0.5	update_cloudrun_sql.sh	CONFIRMED
Inventory Internal IP	10.160.0.6	update_cloudrun_sql.sh	CONFIRMED
Payment Internal IP	10.160.0.7	update_cloudrun_sql.sh	CONFIRMED
Cloud SQL Public IP	34.14.131.84	deploy_gke.sh	CONFIRMED

25. Command Reference

Command (abbreviated)	Purpose	Source
<code>docker buildx build --platform linux/amd64 ...</code>	Builds cross-platform Docker image and pushes to Artifact Registry	<code>deploy_phase1.sh</code>
<code>gcloud auth configure-docker asia-south1-docker.pkg.dev</code>	Configures Docker to authenticate with Artifact Registry	<code>deploy_phase1.sh</code>
<code>gcloud run deploy syntrix-cart ...</code>	Deploys container as Cloud Run service	<code>deploy_phase1.sh</code>
<code>gcloud run services update ... --add-cloudsql-instances=...</code>	Adds Cloud SQL connector to existing Cloud Run service	<code>update_cloudrun_sql.sh</code>
<code>gcloud run jobs create syntrix-seed ...</code>	Creates a Cloud Run Job for database seeding	<code>setup_seed.sh</code>
<code>gcloud run jobs execute syntrix-seed --wait</code>	Executes seed job and waits for completion	<code>setup_seed.sh</code>
<code>gcloud container clusters create-auto syntrix-gke ...</code>	Creates GKE Autopilot cluster	<code>setup_gke_cluster.sh</code>
<code>gcloud container clusters get-credentials syntrix-gke ...</code>	Gets kubectl credentials for GKE cluster	<code>deploy_gke.sh</code>
<code>kubectl create secret generic db-secret ...</code>	Creates Kubernetes secret for database URL	<code>deploy_gke.sh</code>
<code>kubectl apply -f -</code> (inline YAML)	Applies Deployment + Service manifests for order and payment	<code>deploy_gke.sh</code>
<code>gcloud sql instances create syntrix-postgres ...</code>	Creates Cloud SQL PostgreSQL instance	<code>setup_cloudsql.sh</code>
<code>gcloud secrets create database-url ...</code>	Creates Secret Manager secret	<code>setup_cloudsql.sh</code>
<code>gcloud compute instances create syntrix-inventory ...</code>	Creates Compute Engine VM	<code>deploy_inventory.sh</code>
<code>gcloud compute firewall-rules create allow-inventory-internal ...</code>	Creates internal firewall rule for inventory	<code>deploy_inventory.sh</code>
<code>gcloud logging read ... --format=json --limit=2000</code>	Reads logs from Cloud Logging (used by streamer)	<code>realtime_log_streamer.py</code>
<code>bq load --source_format=NEWLINE_DELIMITED_JSON ...</code>	Loads NDJSON data into BigQuery	<code>realtime_log_streamer.py</code>
<code>gcloud beta monitoring channels create ...</code>	Creates email notification channel	<code>create_alerts.sh</code>
<code>gcloud alpha monitoring policies create ...</code>	Creates alert policy from JSON file	<code>create_alerts.sh</code>
<code>gcloud run services add-iam-policy-binding ... allUsers ...</code>	Grants public access to Cloud Run service	<code>deploy_phase1.sh</code>

26. Design Decisions & Trade-offs

Decision	Reason	Alternative	Trade-off	Status
Cloud Run for Gateway/Catalog/Cart	Stateless, fast scaling, scale-to-zero	GKE for all services	Cost savings vs. always-on infra	CONFIRMED
GKE for Order/Payment	Complex orchestration, persistent pods	Cloud Run	Higher cost but better for multi-service calls	INFERRED
Compute Engine for Inventory	Legacy simulation, third resource type	Cloud Run	Demonstrates VM-based workloads	INFERRED
Three compute platforms	Demonstrates multi-platform observability	Single platform	Complexity vs. hackathon differentiation	CONFIRMED
Custom Log Streamer	Control over ETL, schema sanitization	Native Log Sink	More work but cleaner BigQuery schema	INFERRED
API Polling (10s interval)	Simple implementation, no infrastructure	Pub/Sub or streaming	Latency vs. simplicity	CONFIRMED
NDJSON format	BigQuery native format, streaming-compatible	CSV or Avro	Standard choice for BigQuery loading	CONFIRMED
BigQuery for analytics	Serverless SQL, Looker integration	Cloud SQL analytics	Better for large-scale log analysis	CONFIRMED
Looker Studio	Free, native BigQuery connector	Grafana, DataStudio	Zero cost, GCP native	CONFIRMED
Cloud SQL (shared DB)	Simplicity for hackathon	Per-service databases	Coupling vs. speed of development	INFERRED
Pub/Sub events	Event-driven architecture demonstration	Direct HTTP calls only	Adds async capability but no subscribers yet	CONFIRMED
Email alerting	Simple, no extra infrastructure	Slack, PagerDuty	Sufficient for hackathon demo	INFERRED
python-json-logger	Native JSON output, Cloud Logging compatible	Custom formatter	Industry standard for GCP	CONFIRMED
FastAPI	Async support, auto-docs, modern Python	Flask, Django	Best fit for microservices + async HTTP	INFERRED
GKE Autopilot	No node management, pay-per-pod	GKE Standard	Simpler, cost-effective for small workloads	CONFIRMED

27. Project Limitations

- Log Streamer polling introduces 10-second delay — not true real-time
- No deduplication guarantee — timestamp increment by 1 nanosecond may miss or duplicate edge cases
- Pub/Sub topics have publishers but no subscribers — events are published but not consumed
- Shared database across all services creates coupling (anti-pattern for true microservices)
- Database password hardcoded in deployment scripts
- allUsers invoker on all Cloud Run services (should only be on gateway)
- Cloud SQL accessed via public IP from GKE (should use Auth Proxy)
- No HTTPS/TLS between internal services (VPC traffic is unencrypted)
- BigQuery views are not materialized — query cost on every dashboard refresh
- No CI/CD pipeline — all deployments are manual shell scripts
- No health check probes configured in Kubernetes manifests (no readiness/liveness probes)
- No resource requests/limits defined in GKE Deployments
- Alembic configured but no migration scripts generated — schema is created by seed.py directly
- Cost optimization rules are simple threshold-based — no ML or actual billing data
- GCE logs may need manual injection (inject_gce_logs.sh) if container stdout is not captured

28. What Is Actually Implemented vs Conceptual

Component	Status	Evidence
FastAPI microservices (6 services)	FULLY IMPLEMENTED	Source code verified
Docker containerization	FULLY IMPLEMENTED	Dockerfile, docker-compose.yml
Cloud Run deployment (3 services + 1 job)	FULLY IMPLEMENTED	Deploy scripts verified
GKE Autopilot cluster + 2 deployments	FULLY IMPLEMENTED	K8s manifests in deploy_gke.sh
Compute Engine VM	FULLY IMPLEMENTED	deploy_inventory.sh
Cloud SQL PostgreSQL	FULLY IMPLEMENTED	setup_cloudsql.sh
Structured JSON logging	FULLY IMPLEMENTED	shared/logging/logger.py
Distributed tracing (ContextVars)	FULLY IMPLEMENTED	shared/middleware/tracing.py
Log Streamer Daemon	FULLY IMPLEMENTED	realtime_log_streamer.py
BigQuery dataset + raw_logs table	FULLY IMPLEMENTED	Script evidence + raw_logs.ndjson
BigQuery analytical views (6)	FULLY IMPLEMENTED	bq_views.sql

Component	Status	Evidence
Cloud Monitoring alert policies (5)	FULLY IMPLEMENTED	create_alerts.sh, fix_alerts.py
Notification channels (7 email)	FULLY IMPLEMENTED	create_alerts.sh
Pub/Sub topics (3)	PARTIALLY IMPLEMENTED	Publishers exist; no subscribers
Looker Studio dashboard	DESIGN COMPLETED	1143-line design doc; actual dashboard in GCP console
Cost optimization	IMPLEMENTED (SQL-based)	Custom logic, not GCP Recommender
Frontend SPA	FULLY IMPLEMENTED	frontend/ directory
Simulation/chaos engineering	FULLY IMPLEMENTED	Developer Control Panel + simulation_config
Traffic simulator	FULLY IMPLEMENTED	Frontend JS + simulate_traffic.sh
Secret Manager	FULLY IMPLEMENTED	setup_cloudsql.sh
CI/CD pipeline	NOT IMPLEMENTED	Manual scripts only
Cloud Functions	NOT IMPLEMENTED	No evidence
Native Log Sinks	NOT IMPLEMENTED	Custom streamer used instead
Workload Identity	NOT IMPLEMENTED	Direct credentials used

29. Presentation Talking Points

30-Second Elevator Pitch

"CloudPulse is a cloud-native observability platform that monitors a microservices e-commerce application deployed across Cloud Run, GKE, and Compute Engine. We built a custom log pipeline that streams structured JSON logs from all three environments into BigQuery, where SQL-based analytics power Looker Studio dashboards for real-time performance monitoring, error analysis, and cost optimization recommendations — with Cloud Monitoring alerts for proactive incident detection."

1-Minute Explanation

"We built CloudMart, a realistic e-commerce platform with 6 microservices — a FastAPI gateway, catalog, cart, order, payment, and inventory services. To demonstrate multi-platform observability, we deliberately deployed these across three compute platforms: Cloud Run for stateless services, GKE for complex orchestration, and Compute Engine for legacy workloads. Every service emits structured JSON logs with distributed tracing IDs. A custom log streamer daemon polls Cloud Logging every 10 seconds and loads data into BigQuery. Six analytical views transform raw logs into latency analysis, error tracking, usage monitoring, performance KPIs, and cost optimization recommendations. These feed into a 6-sheet Looker Studio dashboard. We also configured 5 Cloud Monitoring alert policies with team-member-specific email notifications. The application includes a built-in chaos engineering panel that can simulate payment failures, inventory shortages, and database slowdowns to generate organic error telemetry."

2-Minute Architecture Walkthrough

"Let me walk you through the architecture. A user visits our CloudMart web portal, which is served by a FastAPI gateway running on Cloud Run. The gateway proxies API requests to five backend services. Catalog and Cart run on Cloud Run for stateless, auto-scaling workloads. Order and Payment run on GKE Autopilot for persistent orchestration — the Order service coordinates a multi-step checkout flow calling Cart, Inventory, and Payment services. Inventory runs on Compute Engine, simulating a legacy VM workload. All services connect to a shared Cloud SQL PostgreSQL database and publish events to Pub/Sub topics.

On the observability side, every service uses a shared logging library that formats stdout as structured JSON with timestamps, severity, service identity, request IDs, trace IDs, and latency metrics. These logs flow automatically into Cloud Logging. Our custom Log Streamer daemon polls Cloud Logging via the gcloud CLI every 10 seconds, sanitizes the data, and batch-loads it as NDJSON into BigQuery. Six SQL views perform ETL: a base cleaning view extracts application fields, then specialized views calculate latency percentiles, error rates, usage patterns, and cost optimization recommendations. Looker Studio connects to these views for a 6-sheet dashboard covering executive overview, performance, errors, infrastructure, cost optimization, and developer deep-dive. Five Cloud Monitoring alert policies watch for 5xx errors on Cloud Run, high CPU on GKE and GCE, BigQuery query slowness, and pipeline errors — each routed to the responsible team member via email."

30. Judge Q&A

Anticipated judge questions with evidence-backed answers:

Q: Why three different compute platforms?

Answer: To demonstrate multi-platform observability, which is the core challenge in real enterprise environments. Cloud Run for stateless/auto-scaling, GKE for orchestration-heavy workloads, Compute Engine for legacy/stateful. Each produces different `resource.type` in Cloud Logging, which our analytics pipeline handles.

Follow-up: Our BigQuery views use CASE statements on `resource.type` to normalize service names across platforms.

Q: Why a custom log streamer instead of native Log Sinks?

Answer: Native Log Sinks export raw Cloud Logging format with deeply nested schemas that cause BigQuery schema management issues. Our custom streamer sanitizes field names (replacing dots/slashes with underscores), stringifies nested JSON payloads, and uses `ALLOW_FIELD_ADDITION` for schema flexibility.

Follow-up: We may have initially tried Log Sinks and pivoted due to schema issues — the sanitization code suggests this.

Q: How does distributed tracing work across different platforms?

Answer: We use Python ContextVars to store `request_id` and `trace_id`. The gateway generates or extracts these from headers (X-Request-Id, X-Trace-Id) and injects them into all downstream HTTP calls. Every service extracts the same headers via TracingMiddleware. The logger automatically adds `logging.googleapis.com/trace` for Cloud Logging trace correlation.

Follow-up: This means a single checkout request generates 6+ correlated log entries across Cloud Run, GKE, and GCE.

Q: What happens if one service fails?

Answer: The Order service implements compensating transactions: if Payment fails, it releases the reserved inventory and marks the order as FAILED. The gateway returns 502 Bad Gateway for upstream errors. Each failure generates structured error logs with the specific reason (`payment_failed`, `inventory_insufficient`, `cart_not_found`).

Follow-up: These error events appear in the `looker_error_analysis` view and can trigger Cloud Monitoring alerts.

Q: How is cost optimization handled?

Answer: Via custom SQL logic in BigQuery, not Google Cloud Recommender. The `looker_cost_optimization_recommendations` view analyzes the last 24 hours of traffic per compute platform. Low-traffic GKE services get recommended for Cloud Run migration (scale-to-zero). Low-traffic GCE VMs get downsize recommendations. High-traffic Cloud Run services get GKE migration recommendations for sustained

use discounts.

Follow-up: The view uses MAX(timestamp) as reference instead of CURRENT_TIMESTAMP, so it works with historical data.

Q: How do alerts work? How do you avoid alert fatigue?

Answer: Five alert policies monitor different infrastructure layers with team-member-specific routing. Cloud Run 5xx alerts go to the Cloud Run team, GKE CPU to the Kubernetes team, GCE CPU to the VM team. Thresholds are set with duration windows (60-180 seconds) to avoid false positives from momentary spikes.

Follow-up: In production, we would add suppression rules, severity-based routing, and PagerDuty integration.

Q: What is the latency of the observability pipeline?

Answer: The log streamer polls every 10 seconds, so the maximum pipeline latency is ~10-20 seconds from log generation to BigQuery availability. Looker Studio refreshes on user interaction. Cloud Monitoring alerts have their own evaluation windows (60-180 seconds) independent of the log pipeline.

Follow-up: For production, we would use Log Sinks or Pub/Sub for near-real-time streaming.

Q: How would this move to production?

Answer: Key changes: (1) Replace custom streamer with native Log Sinks or Dataflow pipeline, (2) Implement Cloud SQL Auth Proxy for all platforms, (3) Add CI/CD with Cloud Build, (4) Restrict IAM to least privilege, (5) Add Kubernetes health probes and resource limits, (6) Enable BigQuery partitioning and clustering, (7) Replace email alerts with PagerDuty/Slack, (8) Add per-service databases, (9) Implement rate limiting and circuit breakers.

Follow-up: The architecture is sound; the gaps are operational maturity items typical of hackathon projects.

Q: What is custom vs. native Google Cloud functionality?

Answer: Custom: Log Streamer Daemon, BigQuery ETL views, cost optimization SQL rules, FastAPI gateway proxy, simulation/chaos engineering system, structured logging formatter, distributed tracing via ContextVars. Native GCP: Cloud Run, GKE, Compute Engine, Cloud SQL, Cloud Logging (ingestion), Cloud Monitoring (alerts), BigQuery (storage/compute), Looker Studio (visualization), Artifact Registry, Pub/Sub, Secret Manager, IAM.

Follow-up: The custom components fill gaps where native GCP services don't directly solve our specific needs.

31. Personal Learning Roadmap

Study guide organized by topic, connecting each concept to our actual implementation:

1. Application Architecture (FastAPI Microservices)

- **What to Know:** How our 6-service architecture works, how the gateway proxies requests, how services communicate via HTTP
- **Our Project Uses:** Our project uses: FastAPI, httpx async client, SQLAlchemy ORM, shared libraries (logging, middleware, models)
- **Likely Judge Question:** Explain how a checkout request flows through Gateway → Order → Cart + Inventory + Payment

2. Docker & Containerization

- **What to Know:** How a single parameterized Dockerfile builds all services, why cross-platform builds are needed
- **Our Project Uses:** Single Dockerfile with SERVICE_NAME build arg, python:3.12-slim base, 2 Uvicorn workers
- **Likely Judge Question:** Why does our build use --platform linux/amd64?

3. Artifact Registry

- **What to Know:** Where Docker images are stored, how authentication works, image tagging strategy
- **Our Project Uses:** syntrix-cloudmart repository in asia-south1-docker.pkg.dev, images tagged with semver (1.0.0, 1.0.2)
- **Likely Judge Question:** How does a container image get from your laptop to Cloud Run?

4. Cloud Run

- **What to Know:** Serverless containers, scale-to-zero, Cloud SQL connectors, VPC egress, IAM invoker bindings
- **Our Project Uses:** Gateway + Catalog + Cart as services; Seed as a Job; VPC egress for internal service communication
- **Likely Judge Question:** Why does the gateway need --vpc-egress=all-traffic?

5. GKE (Autopilot)

- **What to Know:** Kubernetes Deployments, Pods, Services, Secrets, LoadBalancer types, Autopilot vs Standard
- **Our Project Uses:** syntrix-gke Autopilot cluster, Order + Payment Deployments, internal LoadBalancer Services, db-secret
- **Likely Judge Question:** Why Autopilot instead of Standard GKE?

6. Compute Engine

-
- **What to Know:** VM instances, startup scripts, metadata server, firewall rules, Docker on VMs
 - **Our Project Uses:** e2-micro VM running inventory container, metadata for DB_URL, http-server tag, internal firewall
 - **Likely Judge Question:** Why not just use Cloud Run for the inventory service?

7. Cloud SQL

- **What to Know:** Managed PostgreSQL, tiers, connection methods (Auth Proxy, direct IP, private IP)
- **Our Project Uses:** syntrix-postgres (db-f1-micro), connected via Auth Proxy (Cloud Run) and direct IP (GKE, GCE)
- **Likely Judge Question:** What are the security implications of using public IP?

8. Cloud Logging

- **What to Know:** Structured logging, resource types, log severity, trace correlation, log queries
- **Our Project Uses:** All services emit JSON to stdout; Cloud Logging auto-ingests from all platforms
- **Likely Judge Question:** How does Cloud Logging know which service a log came from?

9. Log Streamer & BigQuery Pipeline

- **What to Know:** Custom polling daemon, NDJSON format, batch loading, schema evolution, key sanitization
- **Our Project Uses:** realtime_log_streamer.py: polls every 10s, sanitizes keys, batch loads via bq load
- **Likely Judge Question:** Why not use a native Log Sink to BigQuery?

10. BigQuery Analytics

- **What to Know:** Views, SQL aggregations, APPROX_QUANTILES, SAFE_DIVIDE, TIMESTAMP_TRUNC, JSON_EXTRACT_SCALAR
- **Our Project Uses:** 6 views: cleaned_logs, latency_analysis, error_analysis, usage_monitoring, performance_kpis, cost_optimization
- **Likely Judge Question:** Explain what P90 latency means and how we calculate it

11. Cloud Monitoring & Alerting

- **What to Know:** Metric types, alert policies, conditions, aggregation, notification channels, incidents
- **Our Project Uses:** 5 alert policies covering Cloud Run, GKE, GCE, BigQuery, and pipeline errors
- **Likely Judge Question:** What is the difference between alignmentPeriod and duration in an alert policy?

12. Looker Studio

- **What to Know:** Data sources, calculated fields, dimensions vs. metrics, chart types, multi-sheet dashboards
- **Our Project Uses:** 6-sheet dashboard design documented in 1143-line markdown guide, connected to BigQuery views
- **Likely Judge Question:** How does Looker Studio connect to BigQuery?

13. FinOps / Cost Optimization

- **What to Know:** Right-sizing, scale-to-zero, sustained use discounts, traffic analysis
- **Our Project Uses:** SQL-based rules in `looker_cost_optimization_recommendations` view
- **Likely Judge Question:** How do you determine if a GKE workload should migrate to Cloud Run?

14. IAM & Security

- **What to Know:** Service accounts, roles, bindings, Secret Manager, principle of least privilege
- **Our Project Uses:** `cloudrun-app-sa` with `cloudsql.client` + `secretmanager.secretAccessor`, `allUsers` invoker
- **Likely Judge Question:** What security improvements would you make for production?

15. End-to-End Architecture

- **What to Know:** How everything connects: `user` → `app` → `logs` → `analytics` → `dashboards` → `alerts`
- **Our Project Uses:** Complete data flow from browser click to Looker Studio chart update
- **Likely Judge Question:** Walk me through what happens when a user clicks 'Confirm Order'

32. Final Cheat Sheet

Quick-reference summary of the entire project:

Question	Answer
What is CloudPulse?	Cloud-native observability platform for multi-platform microservices
What is CloudMart?	E-commerce application that generates the log telemetry
How many services?	6 (Gateway + Catalog + Cart + Order + Payment + Inventory)
How many platforms?	3 (Cloud Run + GKE Autopilot + Compute Engine)
Database?	Cloud SQL PostgreSQL 15 (db-f1-micro)
How are logs collected?	JSON stdout → Cloud Logging → Custom Streamer → BigQuery
How fast is the pipeline?	~10-20 second latency (polling interval)
How many BigQuery views?	6 (cleaned + latency + errors + usage + KPIs + cost)
How many alert policies?	5 (Cloud Run 5xx, GKE CPU, GCE CPU, BQ time, Pipeline errors)
How many team members?	8 (7 notification channels configured)
What makes it unique?	3-platform observability + custom ETL + chaos engineering + FinOps SQL
Key limitation?	10-second polling delay, no native Log Sinks, shared database
Production gap?	No CI/CD, hardcoded secrets, broad IAM, no health probes

Appendix

A. Important Files Reference

File	Purpose
gateway/main.py	FastAPI gateway — API proxy, simulation endpoints, frontend serving
services/*/main.py	5 backend microservice entry points
shared/logging/logger.py	CustomJsonFormatter with Cloud Logging integration
shared/middleware/tracing.py	TracingMiddleware for distributed tracing
shared/models/models.py	SQLAlchemy models (8 tables)
shared/utis/database.py	Database connection factory
Dockerfile	Multi-service parameterized container build
docker-compose.yml	Local development orchestration
database/seed_data/seed.py	Database schema creation + 50 product seed
deploy_phase1.sh	Cloud Run deployment (cart + gateway)
deploy_gke.sh	GKE deployment (order + payment) with K8s manifests
deploy_inventory.sh	Compute Engine deployment (inventory)
setup_cloudsql.sh	Cloud SQL provisioning + IAM + Secret Manager
setup_gke_cluster.sh	GKE Autopilot cluster creation
setup_seed.sh	Cloud Run Job for database seeding
update_cloudrun_sql.sh	Gateway service URL + Cloud SQL connector update
realtime_log_streamer.py	Log Streamer Daemon (Cloud Logging → BigQuery)
export_logs.sh	Bulk log export + BigQuery initial load
prepare_bq_json.py	JSON → NDJSON sanitizer for BigQuery
bq_views.sql	6 BigQuery analytical views
create_alerts.sh	5 Cloud Monitoring alert policies + 7 notification channels
fix_alerts.py	Alert policy fix (aggregation parameters)
simulate_traffic.sh	CLI traffic generator
Looker_Dashboard_Design.md	1143-line Looker Studio dashboard design guide
frontend/index.html	SPA main page with Developer Control Panel
frontend/app.js	Core application logic (routing, API calls)
frontend/admin.js	Developer Control Panel + Traffic Simulator

B. Project Structure Tree

SYNTRIX/
■ gateway/main.py # API Gateway
■ services/
■ ■ cart/main.py # Cart Service
■ ■ catalog/main.py # Catalog Service
■ ■ order/main.py # Order Service
■ ■ payment/main.py # Payment Service
■ ■ inventory/main.py # Inventory Service
■ shared/
■ ■ logging/logger.py # Structured JSON logger
■ ■ middleware/tracing.py # Distributed tracing
■ ■ models/models.py # SQLAlchemy models
■ ■ utils/database.py # DB connection
■ frontend/
■ ■ index.html # SPA main page
■ ■ app.js # App logic
■ ■ admin.js # Dev panel + traffic sim
■ ■ styles.css # Glassmorphism CSS
■ database/
■ ■ seed_data/seed.py # DB seed script
■ ■ migrations/ # Alembic (configured)
■ ■ alembic.ini
■ alerts/ # Alert policy JSONs
■ Dockerfile # Multi-service container
■ Dockerfile.seed # Seed job container
■ docker-compose.yml # Local dev environment
■ requirements.txt # Python dependencies
■ deploy_phase1.sh # Cloud Run deploy
■ deploy_gke.sh # GKE deploy + K8s manifests
■ deploy_inventory.sh # Compute Engine deploy
■ setup_cloudsql.sh # Cloud SQL setup
■ setup_gke_cluster.sh # GKE cluster creation
■ setup_seed.sh # Cloud Run Job seed
■ update_cloudrun_sql.sh # Gateway + Cloud SQL update
■ realtime_log_streamer.py # Log Streamer Daemon
■ export_logs.sh # Bulk log export
■ prepare_bq_json.py # NDJSON sanitizer
■ bq_views.sql # BigQuery analytics views
■ create_alerts.sh # Alert policies + channels

■■■	fix_alerts.py	# Alert fix script
■■■	simulate_traffic.sh	# Traffic generator
■■■	Looker_Dashboard_Design.md	# Dashboard design guide
■■■	README.md	# Project documentation

End of Document — Generated by SYNTRIX Project Analysis Tool